RECAP SEMESTRE 1 — Cheat sheet e domande frequenti

> Tutto quello che serve sapere del semestre 1, in un posto solo. Nel semestre 2 **ogni giorno** costruiremo sopra queste basi: se un punto qui è nebbioso, ripassatelo **adesso**, non durante il lab.
>
> Per le parole di base (client, server, porta, JSON...) c'è `CONCETTI_BASE.md`.

Indice

1. Il terminale
2. Virtual environment e dipendenze
3. Python difensivo: type hint ed eccezioni
4. pytest: il primo test e il pattern AAA
5. Testare gli errori: `pytest.raises`
6. Un test, tanti casi: `parametrize` e `nullcontext`
7. Misurare: coverage e branch coverage
8. Isolare: il mock e la regola del path
9. Git: le basi
10. Git Flow
11. Continuous Integration con GitHub Actions
12. Black box: testare una CLI con `subprocess`
13. La mappa: white box, black box e piramide dei test
14. Altri strumenti visti
15. Domande frequenti

---

1. Il terminale

Cosa voglio fare	Mac / Linux	Windows (PowerShell)
Dove sono?	`pwd`	`Get-Location` (o `pwd`)
Cosa c'è qui?	`ls`	`dir` (o `ls`)
Entrare in una cartella	`cd nome`	`cd nome`
Tornare indietro di uno	`cd ..`	`cd ..`
Creare una cartella	`mkdir nome`	`mkdir nome`
Creare un file vuoto	`touch file.py`	`ni file.py`
Fermare un programma	`CTRL+C`	`CTRL+C`

- **TAB** completa i nomi: se non completa, avete sbagliato a scrivere o siete nella cartella sbagliata.
- **Freccia su** richiama i comandi precedenti.

---

2. Virtual environment e dipendenze

```
python -m venv venv                 # crea il "recinto" (una volta sola per progetto)

source venv/bin/activate            # Mac/Linux: lo accende
.\venv\Scripts\Activate.ps1         # Windows: lo accende

pip install -r requirements.txt     # installa la "lista della spesa"
pip freeze > requirements.txt       # salva la lista di ciò che è installato
```

- Se vedete `(venv)` all'inizio della riga, il recinto è acceso.
- **Ogni terminale nuovo** va attivato di nuovo.
- La cartella `venv/` **non** va mai su Git (va nel `.gitignore`) e **non** va nello zip di consegna.

---

3. Python difensivo: type hint ed eccezioni

```
def valida_password(password: str) -> bool:      # type hint: "entra una stringa, esce un
booleano"
    if len(password) < 8:
        raise ValueError("Password troppo corta")   # freno di emergenza: si ferma TUTTO
    return True
```

- **Mai "ingoiare" gli errori** (`return False` o `return None` quando qualcosa va storto): il programma continua zoppicando e corrompe i dati. Si usa `raise`.

- **Leggere un traceback:** dal **basso verso l'alto**. L'ultima riga dice **cosa** è successo (`ValueError: ...`), le righe sopra dicono **dove** (file e numero di riga).

---

4. pytest: il primo test e il pattern AAA

```
# test_carrello.py      <- il file DEVE iniziare con test_
from carrello import calcola_totale

def test_totale_due_prodotti():          # <- la funzione DEVE iniziare con test_
    # Arrange: preparo i dati
    prodotti = [10.0, 5.5]

    # Act: eseguo UNA azione
    totale = calcola_totale(prodotti)

    # Assert: verifico il risultato
    assert totale == 15.5
```

```
pytest              # lancia tutti i test
pytest -v           # verboso: un test per riga
pytest test_x.py    # solo un file
pytest -k totale    # solo i test che contengono "totale" nel nome
pytest -s           # mostra i print
```

---

5. Testare gli errori: `pytest.raises`

```
import pytest

def test_password_corta_viene_rifiutata():
    with pytest.raises(ValueError, match="troppo corta"):
        valida_password("123")
```

- **Happy path:** il caso in cui tutto va bene. **Sad path:** il caso in cui qualcosa va storto. Vanno testati **entrambi**.
- Il test **passa** se l'eccezione viene sollevata. Se il codice **non** esplode, il test è **rosso**.

---

6. Un test, tanti casi: `parametrize` e `nullcontext`

```
import pytest
from contextlib import nullcontext as does_not_raise

@pytest.mark.parametrize("peso, zona, aspettativa, atteso", [
    (1.5, "EU",   does_not_raise(),                       5.0),   # happy path
    (3.0, "USA",  does_not_raise(),                       25.0),  # happy path
    (-1,  "EU",   pytest.raises(ValueError, match="positivo"), None),  # sad path
    (2.0, "ASIA", pytest.raises(ValueError, match="supportata"), None), # sad path
])
def test_spedizione(peso, zona, aspettativa, atteso):
    with aspettativa:
        assert calcola_spedizione(peso, zona) == atteso
```

- Copiare un test cambiando solo i numeri = **debito tecnico**. Si usa una **tabella di dati**.
- `does_not_raise()` (cioè `nullcontext`) = "qui non deve succedere niente". `pytest.raises(...)` = "qui deve esplodere".
- **Boundary Value Analysis:** i bug vivono sui **confini**. Se la regola è "sotto i 2 kg", testate 1.99, **2.0** e 2.01, non 1 e 5.

---

7. Misurare: coverage e branch coverage

```
pip install pytest-cov
pytest --cov=nome_modulo --cov-report=term-missing              # righe eseguite
pytest --cov=nome_modulo --cov-branch --cov-report=term-missing  # anche i rami True/False
```

- La colonna `Missing` dice le righe (o i rami, es. `15->exit`) che nessun test ha mai eseguito.
- **Line coverage** = quante righe sono state lette. **Branch coverage** = per ogni `if`, è stato provato sia il caso vero sia il caso falso? È la metrica seria.
- Attenzione: **100% di coverage non vuol dire 0 bug**. Vuol dire che ogni riga è stata eseguita, non che ogni risultato è stato verificato.

---

8. Isolare: il mock e la regola del path

Un test che dipende da rete, orologio o casualità è **flaky** (instabile). Il mock sostituisce il pezzo imprevedibile con un sosia che risponde quello che diciamo noi.

```
from unittest.mock import patch

# gioco.py contiene:  import random  ...  random.randint(1, 6)

@patch("gioco.random.randint")          # <- DOVE VIENE USATO, non "random.randint"
def test_vittoria_con_il_sei(mock_dado):
    mock_dado.return_value = 6          # trucco il dado
    assert gioca() == "vittoria"

# Alternativa con il context manager:
def test_sconfitta_con_l_uno():
    with patch("gioco.random.randint") as mock_dado:
        mock_dado.return_value = 1
        assert gioca() == "sconfitta"
```

- `return_value` : cosa risponde il finto.
- `side_effect = Eccezione(...)` : il finto **esplode** (per simulare un guasto).
- ⚠️ **Regola d'oro:** si intercetta l'oggetto **dove viene usato**, cioè `modulo_che_lo_usa.nome` .
- Con più `@patch` impilati, i mock arrivano come parametri **dal basso verso l'alto**.

---

9. Git: le basi

```
 Working Directory  ──git add──►  Staging Area  ──git commit──►  Repository
 (il tavolo)                      (la scatola)                    (la cassaforte)
```

```
git init                                # inizia a tracciare questa cartella
git status                              # il radar: usatelo sempre
git add file.py                         # nella scatola (git add . = tutto)
git commit -m "feat: calcolo spedizione"  # sigillo con un messaggio che spiega il PERCHÉ
git log --graph --oneline --all         # la mappa dei commit

git checkout -b feature/login           # nuovo branch + ci entro (anche: git switch -c)
git checkout main                       # torno su main
git merge feature/login                 # porto le modifiche DENTRO il branch in cui sono

git remote add origin <URL>             # collego il repo a GitHub
git push -u origin main                 # la prima volta; poi basta "git push"
git pull                                # scarico le novità dal remoto
```

- **Conventional Commits:** `feat:` nuova funzionalità · `fix:` correzione · `test:` test · `refactor:` riorganizzo senza cambiare comportamento · `docs:` documentazione · `chore:` manutenzione varia.
- `.gitignore` **:** l'elenco di ciò che **non** deve finire su Git: `venv/` , `__pycache__/` , `.env` .

---

10. Git Flow

Branch	A cosa serve	Si stacca da → si fonde in
`main`	il codice **in produzione**. Sacro: niente push diretti, solo Pull Request	—
`develop`	il cantiere: le feature finite si integrano qui	—
`feature/...`	una funzionalità nuova	`develop` → `develop`
`release/...`	collaudo prima del rilascio: solo correzioni	`develop` → `main` + `develop`
`hotfix/...`	l'ambulanza: bug urgente in produzione	`main` → `main` + `develop`

11. Continuous Integration con GitHub Actions

Il vostro PC è un ambiente "sporco". La CI esegue i test su un server **pulito**, a ogni push.

```
# .github/workflows/ci.yml        <- percorso ESATTO
name: CI
on: [push]                        # quando parte
jobs:
  test:
    runs-on: ubuntu-latest        # su quale macchina
    steps:
      - uses: actions/checkout@v7          # scarica il codice
      - uses: actions/setup-python@v7      # installa Python
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt
      - run: pytest
```

- **YAML è spietato con l'indentazione:** solo spazi, mai TAB.
- **Pallino verde** = test passati. **Pallino rosso** = si aprono i **log** e si legge l'errore. Mai andare a tentativi.
- Nel semestre 1 usavamo `checkout@v4` e `setup-python@v5`: funzionano ancora, ma le versioni attuali sono le `@v7`.

12. Black box: testare una CLI con `subprocess`

```
import subprocess, sys

def test_firewall_blocca_ip_in_blacklist():
    risultato = subprocess.run(
        [sys.executable, "firewall.py", "199.0.0.1", "user"],   # come scriverlo nel
terminale
        capture_output=True, text=True,
    )
    assert risultato.returncode == 2                          # exit code
    assert "MINACCIA RILEVATA" in risultato.stderr            # messaggio sul canale giusto
```

Standard POSIX: i canali e gli exit code

Canale / codice	Significato
`stdout`	i risultati andati a buon fine (`print(...)`)
`stderr`	errori e diagnostica (`print(..., file=sys.stderr)`)
exit code `0`	tutto ok
exit code `1`	errore generico (es. parametri mancanti)
exit code `2` (e oltre)	errori specifici (es. regola di business violata)

- Nel programma: `sys.argv` legge gli argomenti, `sys.exit(codice)` termina con un exit code.
- Nei test black box **niente** `import` del programma: siete ispettori esterni.
- `sys.executable` = "lo stesso Python del venv che sta eseguendo i test" (più sicuro di `"python"`).

---

13. La mappa: white box, black box e piramide dei test

	White box	Black box
Cosa guardo	il codice dentro (if, cicli)	solo input → output
Come	`import` , mock, coverage	`subprocess` , HTTP, browser
Pregio	veloce, preciso	resiste ai refactoring
Difetto	si rompe se cambio l'implementazione	più lento, dice "cosa" ma non "dove"

```
          /\        E2E / interfaccia      pochi, lenti, costosi
         /  \
        /----\      Integrazione / API     una via di mezzo
       /      \
      /--------\    Unit                   tanti, veloci, economici
```

---

14. Altri strumenti visti

Strumento	Cosa fa	In una riga
`.env` + `python-dotenv`	tiene i segreti **fuori** dal codice	`load_dotenv()` poi `os.getenv("API_KEY")`; il `.env` va nel `.gitignore`
GitLeaks	cerca password e chiavi finite per sbaglio nella storia di Git	un segreto pushato è **compromesso**: va cambiato, non solo cancellato
`pre-commit`	lancia controlli **prima** di ogni commit	se `black` o `pytest` falliscono, il commit non parte
`black`	formatta il codice in modo uniforme	niente discussioni sullo stile
Mutation testing (`mutmut`)	modifica apposta il codice (es. `>` diventa `>=`) e guarda se i test se ne accorgono	un mutante "sopravvissuto" = un test che manca

15. Domande frequenti

Ambiente

`ModuleNotFoundError: No module named 'pytest'` **(o** `flask`**, o** `requests`**)** Il venv non è attivo in **quel** terminale, oppure non avete fatto `pip install -r requirements.txt`. Controllate che ci sia `(venv)` all'inizio della riga.

`pytest` **dice "no tests ran" / "collected 0 items"** Il file deve chiamarsi `test_qualcosa.py` e le funzioni `def test_qualcosa():`. Controllate anche di essere nella cartella giusta (`pwd`).

Su Windows, `Activate.ps1` **dà un errore rosso sui "criteri di esecuzione"** `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, poi riprovate.

`python` **non trovato su Mac** Su Mac spesso si chiama `python3`. **Dentro** il venv attivo, invece, `python` funziona sempre.

Test

Che differenza c'è tra un test rosso per `AssertionError` **e uno rosso per un'altra eccezione?** `AssertionError` = il codice gira ma il risultato è sbagliato. Un'altra eccezione (`TypeError`, `KeyError`...) = il codice è **esploso** prima di arrivare all'assert. Leggete l'ultima riga del traceback.

Il mio mock non funziona: il test usa ancora la funzione vera Al 99% il path è sbagliato. Si scrive `"modulo_che_usa.nome"`, non il nome della libreria originale. Vedi la sezione 8.

Con due `@patch`**, i parametri sono invertiti** I decoratori si applicano dal basso: il `@patch` più **vicino** alla funzione è il **primo** parametro.

`return_value` **o** `side_effect`**?** `return_value` = "rispondi questo". `side_effect` = "esplodi con questa eccezione" (o "rispondi in sequenza questi valori", se gli date una lista).

Ho il 100% di coverage: ho finito? No. La coverage dice che una riga è stata **eseguita**, non che il risultato è stato **verificato**. Per sapere se i test sono buoni davvero, si usa il mutation testing.

`assert 0.1 + 0.2 == 0.3` **fallisce!** I numeri `float` non sono precisi. Si usa `assert x == pytest.approx(0.3)`. E per i soldi, in un sistema vero, si usa `Decimal` o si conta in centesimi.

Devo testare anche le funzioni "ovvie"? Testate il **comportamento** che conta per il business e i **confini** (Boundary Value Analysis). Un getter banale no; il calcolo di uno sconto sì, sempre.

Git

`git push` **rifiutato ("rejected", "fetch first")** Sul remoto ci sono commit che voi non avete. Prima `git pull`, poi di nuovo `git push`.

Ho committato su `main` **invece che su un branch** Se non avete ancora pushato: `git checkout -b feature/nome` (il branch nuovo si porta dietro il commit). Poi si sistema `main` con calma: chiedete prima di usare comandi distruttivi.

Conflitto di merge (`CONFLICT` **)** Git non sa quale versione tenere. Aprite il file: tra `<<<<<<<` e `>>>>>>>` ci sono le due versioni. Tenete quella giusta, cancellate i segni, poi `git add` e `git commit`.

Ho pushato il file `.env` **con una password** Cancellarlo non basta: è nella storia di Git. La password va considerata **bruciata** e va **cambiata**. Poi `.env` va nel `.gitignore`.

Ho messo `venv/` **su GitHub** Aggiungete `venv/` al `.gitignore`, poi `git rm -r --cached venv` e committate.

CI

La pipeline è rossa ma sul mio PC i test passano È proprio il motivo per cui esiste la CI: il vostro PC ha qualcosa che il server pulito non ha. Di solito è una libreria installata a mano e non scritta in `requirements.txt`. Aprite i log del passo rosso.

La pipeline non parte proprio Il file deve stare **esattamente** in `.github/workflows/` e finire in `.yml`. Controllate l'indentazione: solo spazi.

Black box

`subprocess.run` **non trova il file** Il percorso è relativo alla cartella **da cui lanciate pytest**. Lanciate `pytest` dalla cartella del progetto.

L'exit code è sempre 0 anche quando c'è un errore Nel programma state stampando l'errore ma non chiamate `sys.exit(codice)`. Il sistema operativo legge solo il numero, non i vostri `print`.

`stdout` **è vuoto ma vedo il messaggio** State stampando su `stderr` (o viceversa). Controllate `risultato.stderr`. Ricordatevi `capture_output=True, text=True`.

Documentazione ufficiale

- pytest: https://docs.pytest.org/en/stable/
- unittest.mock, dove fare patch: https://docs.python.org/3/library/unittest.mock.html#where-to-patch

- Coverage.py: https://coverage.readthedocs.io/
- Git, libro ufficiale (in italiano): https://git-scm.com/book/it/v2
- Conventional Commits: https://www.conventionalcommits.org/it/v1.0.0/
- GitHub Actions: https://docs.github.com/en/actions
- subprocess: https://docs.python.org/3/library/subprocess.html
- La piramide dei test (Martin Fowler): https://martinfowler.com/articles/practical-test-pyramid.html